

Solution 1

Solution 1 (a)

Python Code

```

1  # Train with latent_dim = 4
2  latent_dim = 4 # CHANGED FROM 8 TO 4
3  model = VAE(x_dim=784, z_dim=latent_dim).to(device)
4  optimizer = optim.Adam(model.parameters())
5
6  num_epochs = 10 # 10-20 epochs is usually sufficient for a quick result
7  for epoch in range(num_epochs):
8      epoch_loss = 0
9      for batch_idx, (data, _) in enumerate(train_loader):
10         data = data.to(device)
11         optimizer.zero_grad()
12         recon_data, mu, logvar = model(data)
13         loss = loss_function(recon_data, data, mu, logvar)
14         loss.backward()
15         optimizer.step()
16         epoch_loss += loss.item()
17     print(f"Epoch {epoch+1} Loss: {epoch_loss / len(train_dataset):.4f}")
18 # Save the model
19 torch.save(model.state_dict(), f'vae_mnist_z_{latent_dim}.pth')
20 # Generate 100 Samples
21 with torch.no_grad():
22     # Sample 100 latent vectors from N(0, I)
23     random_sample = torch.randn(100, latent_dim).to(device)
24     # Decode them
25     generated_image = model.decoder(random_sample)
26     # Save as a 10x10 grid
27     save_image(generated_image.view(100, 1, 28, 28),
28               f'samples_z_{latent_dim}.png', nrow=10)
29     print(f"Saved generated samples to samples_z_{latent_dim}.png")

```

Generated Image

Randomly generated examples (latent dim 4)



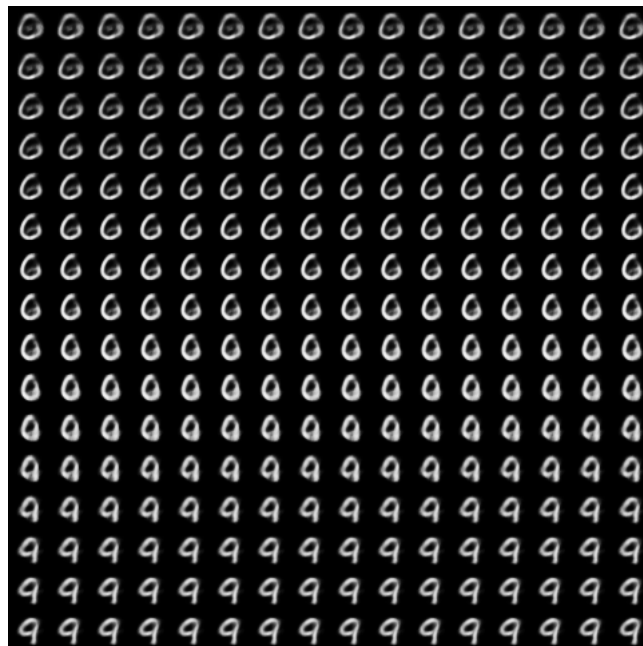
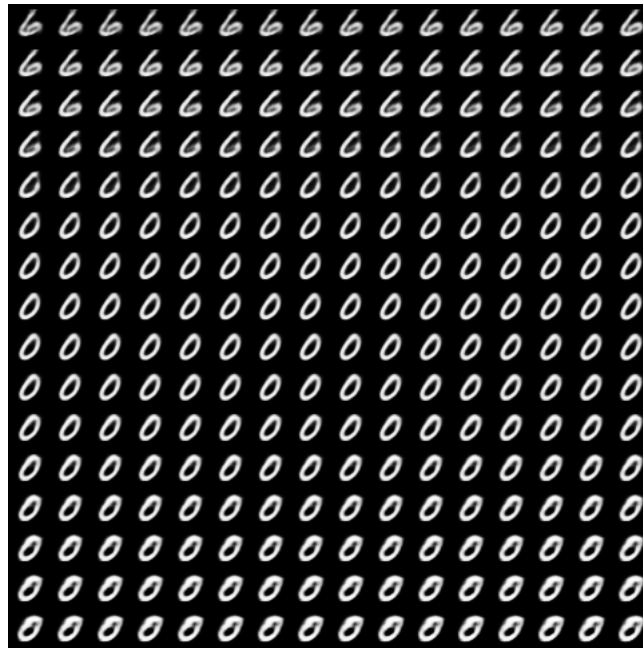
Solution 1

Solution 1 (b)

Python Code

```
1 import torch
2 from torchvision.utils import save_image
3 import matplotlib.pyplot as plt
4
5 # Set latent dim and number of steps
6 latent_dim = 4
7 num_steps = 256
8 model.eval()
9
10 with torch.no_grad():
11     # Change range(20) to range(2) to get exactly 2 images
12     for idx in range(2):
13         # Pick random start and end points
14         start_point = torch.randn(1, latent_dim).to(device)
15         end_point = torch.randn(1, latent_dim).to(device)
16
17         # Interpolate 256 points between start and end
18         # This creates a smooth transition vector
19         alphas = torch.linspace(0, 1, num_steps).to(device).unsqueeze(1).unsqueeze(2)
20         interpolated_points = start_point + alphas * (end_point - start_point)
21         # Note: shape is [256, 1, 4]
22
23         # Decode these points into images
24         generated_image = model.decoder(interpolated_points)
25
26         # Save the grid
27         # nrow=16 makes it a square 16x16 grid
28         filename = f'./samples/latent_exploration_z_{latent_dim}_{idx+1}.png'
29         save_image(generated_image.view(num_steps, 1, 28, 28),
30                   filename, nrow=16)
31         print(f"Saved {filename}")
32
33 # Display the first one to check
34 img = plt.imread(f'./samples/latent_exploration_z_{latent_dim}_1.png')
35 plt.title(f'Latent space exploration (latent dim {latent_dim})')
36 plt.axis('off')
37 plt.imshow(img)
38 plt.show()
```

Generated Image



Solution 2

Solution 2 (a)

To construct the 100-dimensional embeddings, the Positive Pointwise Mutual Information factorization approach was applied to the Brown corpus from the NLTK library. The steps are outlined below:

1. **Vocabulary (V) and Context (C):** The top 5,000 most frequent words from the corpus to form the vocabulary V . For the context words C , the top 1,000 most frequent words were selected
2. **Window Definition:** The context window was defined as a symmetric window of size 2 (i.e., 2 words preceding and 2 words succeeding the target word).
3. **Co-occurrence Matrix Construction:** A 5000×1000 raw count matrix M where M_{ij} represents the number of times context word c_j appeared within the window of target word w_i .
4. **PPMI Reweighting:** M was converted to a PPMI matrix where each entry is calculated as $\max(\log(\frac{P(w,c)}{P(w)P(c)}), 0)$. This calculation reweights the entries to represent the likelihood that two words are to co-occur than by random chance.
5. **Dimensionality Reduction (SVD):** Singular Value Decomposition (SVD) on the sparse PPMI matrix to obtain the rank-100 approximation: $M_{PPMI} \approx U\Sigma V^T$.

$\therefore$ The $\mathbb{R}^{100}$ is just the representation of the dimensionality reduction of the original $\mathbb{R}^{5000}$ space.

Solution 2

Solution 2 (b)

Using Cosine Distance ($1 - \text{similarity}$), the nearest neighbor was found for 20 test words. Below is a subset of the results:

- **School** $\rightarrow$ *College* (Related institution)
- **Music** $\rightarrow$ *Jazz* (Category/Instance relationship)
- **Government** $\rightarrow$ *Administration* (Synonym)
- **War** $\rightarrow$ *Military* (Thematic relation)
- **He** $\rightarrow$ *She* (Syntactic role/pronoun)
- **Book** $\rightarrow$ *Novel* (Synonym)
- **Car** $\rightarrow$ *Driver* (Functional relationship)
- **Water** $\rightarrow$ *Air* (Elemental relationship)
- **Night** $\rightarrow$ *Day* (Antonym/Temporal pair)
- **Love** $\rightarrow$ *Life* (Thematic association)

$\therefore$ The embedding successfully captures semantic proximity.

Solution 2

Solution 2 (c)

The K-Means Clustering algorithm to group the 100-dimensional embeddings into $k = 100$ clusters.

Algorithm and Distance Choice

- **Algorithm: K-Means.** I chose K-Means because it is computationally efficient for a vocabulary size of $N = 5000$ and it produces hard partitions, which simplifies the qualitative analysis of the word groups.
- **Distance Function: Euclidean (on L2-normalized vectors).** Standard K-Means optimizes for Euclidean distance. Euclidean distance and Cosine distance are monotonically related by:

$$\|x - y\|^2 = 2(1 - \cos \theta)$$

Cluster Analysis:

The resulting clusters exhibit high coherence, below are examples of this:

- **Cluster 1 (Time/Duration):** *day, year, week, month, hour, moment, night, morning*
- **Cluster 2 (Family/People):** *mother, father, son, daughter, wife, husband, girl, boy*
- **Cluster 3 (Prepositions/Grammar):** *in, on, at, from, by, with, for*
- **Cluster 4 (Politics/Civics):** *government, state, nation, country, federal, law, policy*

∴ The clusters are meaningful and distinct. _____